

3 A program sorts an array of integers and searches the array for a particular value.

(a) The array of integers, `NumberArray`, stores the following data in the order given:

100 85 644 22 15 8 1

The array is declared and initialised local to the main program.

Write program code to declare and initialise the array.

Save your program as **Question3_J24**.

Copy and paste the program code into part **3(a)** in the evidence document.

[1]

- (b) (i) The following recursive pseudocode function sorts the array into ascending order using an insertion sort and returns the sorted array.

```

DECLARE LastItem : INTEGER
DECLARE CheckItem : INTEGER
DECLARE LoopAgain : BOOLEAN
FUNCTION RecursiveInsertion(IntegerArray : ARRAY[] OF INTEGER,
    NumberElements : INTEGER) RETURNS ARRAY[] OF INTEGER
    IF NumberElements <= 1 THEN
        RETURN IntegerArray
    ELSE
        CALL RecursiveInsertion(IntegerArray, NumberElements - 1)
        LastItem ← IntegerArray[NumberElements - 1]
        CheckItem ← NumberElements - 2
    ENDIF
    LoopAgain ← TRUE
    IF CheckItem < 0 THEN
        LoopAgain ← FALSE
    ELSE
        IF IntegerArray[CheckItem] < LastItem THEN
            LoopAgain ← FALSE
        ENDIF
    ENDIF
    WHILE LoopAgain
        IntegerArray[CheckItem + 1] ← IntegerArray[CheckItem]
        CheckItem ← CheckItem - 1
        IF CheckItem < 0 THEN
            LoopAgain ← FALSE
        ELSE
            IF IntegerArray[CheckItem] < LastItem THEN
                LoopAgain ← FALSE
            ENDIF
        ENDIF
    ENDWHILE
    IntegerArray[CheckItem + 1] ← LastItem
    RETURN IntegerArray
ENDFUNCTION

```

Write the program code for the pseudocode function `RecursiveInsertion()`.

Save your program.

Copy and paste the program code into part **3(b)(i)** in the evidence document.

(ii) Amend the main program to:

- call `RecursiveInsertion()` with the initialised array `NumberArray` and the number of elements as parameters
- output the word 'Recursive'
- output the content of the returned array.

Save your program.

Copy and paste the program code into part **3(b)(ii)** in the evidence document.

[2]

(iii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **3(b)(iii)** in the evidence document.

[1]

(c) The function `RecursiveInsertion()` can be changed to use iteration instead of recursion.

(i) Write program code for the function `IterativeInsertion()` to perform the same processes as `RecursiveInsertion()` but using iteration instead of recursion.

Save your program.

Copy and paste the program code into part **3(c)(i)** in the evidence document.

[4]

(ii) Write program code to amend the main program to also:

- call `IterativeInsertion()` with the original initialised array `NumberArray`
- output the word 'iterative'
- output the content of the returned array.

Save your program.

Copy and paste the program code into part **3(c)(ii)** in the evidence document.

[1]

(iii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **3(c)(iii)** in the evidence document.

[1]

(d) The recursive function `BinarySearch()` takes the parameters:

- `IntegerArray` – an array of integers
- `First` – the index of the first array element
- `Last` – the index of the last array element
- `ToFind` – an integer to search for in the array.

The function uses recursion to perform a binary search for `ToFind` in `IntegerArray`.

The function returns the index where `ToFind` is stored or returns `-1` if `ToFind` is **not** in the array.

(i) Write program code for the recursive function `BinarySearch()`.

Save your program.

Copy and paste the program code into part **3(d)(i)** in the evidence document.

[6]

(ii) Write program code to amend the main program to:

- call `BinarySearch()` with the sorted array and the integer 644 as the search value
- output 'Not found' if 644 is **not** found in the array
- output the index if 644 is found in the array.

Save your program.

Copy and paste the program code into part **3(d)(ii)** in the evidence document.

[2]

(iii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **3(d)(iii)** in the evidence document.

[1]